

VISTA: Optimizing GPU Scheduling through Versatile Locality-Aware Data Sharing

Hajar Falahati*, Negin Mahani*, Adrian Cristal, and Osman Unsal
Barcelona Supercomputing Center (BSC), Barcelona, Spain
{hajar.falahati, neginsadat.nematollahizadeh, adrian.cristal, osman.unsal}@bs.es

Abstract—Graphics Processing Units (GPUs) play a pivotal role in high-performance computing by utilizing the massive parallelism of concurrent thread execution to enhance processing efficiency, known as thread-level parallelism (TLP). However, effectively managing the significant memory access generated by a large number of concurrent threads presents a significant challenge, affecting both performance and energy consumption. However, previous GPU scheduling methods often overlook the significant potential of data sharing among non-adjacent warps or Cooperative Thread Arrays (CTAs), thereby limiting their effectiveness in reducing unnecessary memory accesses through supporting data sharing. Our observation underscores the untapped potential of non-adjacent data sharing, motivating our work to optimize schedules more effectively. In this paper, we present VISTA, a smart locality-aware GPU scheduler that dynamically identifies data locality patterns at both the CTA and warp levels, leveraging this runtime information to make intelligent scheduling decisions and improving memory efficiency and overall performance. VISTA significantly reduces unnecessary memory accesses. Simulation results show a 48.1% improvement in performance and a 51.8% reduction in energy consumption for memory-intensive GPGPU applications, compared to the baseline, with negligible hardware overhead.

Index Terms—GPGPU, Memory Access, Energy Efficiency, Performance, Data Sharing, Scheduling.

I. INTRODUCTION

The rising demand for applications with intensive processing requirements has led to a surge in the utilization of Graphics Processing Units (GPUs). GPUs offer remarkable throughput by leveraging high Thread Level Parallelism (TLP), but deploying general-purpose applications on these processors presents significant memory challenges [1], [2].

While increasing TLP aims to hide latency, it also intensifies simultaneous memory access, leading to cache contention and bandwidth saturation. A significant portion of performance degradation comes from L1D cache misses, which lead to accesses to the L2 cache and DRAM, exacerbating power consumption and latency due to excessive memory hierarchy and interconnection network accesses [3]–[7].

Previous work to tackle memory access challenges in GPUs mainly focuses on refining scheduling mechanisms. Various scheduling policies have been proposed to fine-tune memory access patterns and enhance overall system performance [8]–[23]. These policies typically aim to exploit data locality within the GPU’s memory hierarchy, striving to minimize unnecessary memory accesses and cache misses. However,

existing scheduling methods fail to effectively capture and utilize data sharing among threads. For example, conventional schedulers typically employ round-robin or throttling-based strategies. Meanwhile, contemporary warp or Cooperative Thread Arrays (CTA) scheduling techniques [15], prioritize preserving locality in L1D cache. They achieve this by scheduling neighboring warps or CTAs on the same Streaming Multiprocessors (SM) and granting them execution priority. However, this approach may not fully use the opportunity for data sharing among non-adjacent CTAs or warps (i.e., CTAs or warps with IDs differing by more than 1).

Through our analysis of data-sharing patterns in GPUs, we make three key observations that underscore the potential for improving memory access efficiency. First, memory-intensive GPGPU applications exhibit significant data sharing, 80.4% on average within a single SM (Intra-SM) and 68.2% between different SMs (Inter-SM). The substantial Inter-SM data sharing points to inefficiencies in current CTA scheduling policies, which fail to fully exploit existing data sharing. Second, we find that 95% to 78% of shared data occurs as the distance between SMs (i.e., the difference between two SM IDs) increases from 0 to 40, indicating substantial data sharing between non-adjacent CTAs. Third, we observe that non-adjacent warps account for around 80% of total data sharing within an SM, underscoring the opportunity to optimize memory access by capturing this sharing. However, existing sharing-based schedulers overlook the benefits of non-adjacent data sharing. This highlights an urgent need to explore and exploit data sharing among non-adjacent CTAs and warps to mitigate memory access challenges in GPUs.

We propose VISTA, a smart locality-aware GPU scheduler that dynamically identifies data locality patterns at both the CTA and warp levels using a lightweight learning-based model to decrease unnecessary memory accesses while preserving L1D cache locality.

First, VISTA proactively identifies data locality among CTAs, classifies CTAs based on locality, and assigns them to the SM with the most similar memory behavior. To achieve this, VISTA features a CTA locality tracker that utilizes a lightweight clustering approach to group CTAs based on their memory access patterns. It monitors each CTA’s memory behavior by running sample warps on idle subcores and assigns the CTA to the SM with the most similar memory access patterns (i.e., those that access the same addresses in the same order).

*These authors contributed equally to this work.

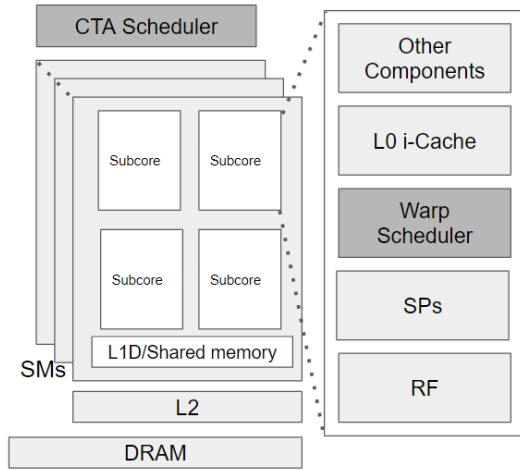


Fig. 1. Baseline GPU architecture.

Second, once CTAs are assigned to the SMs with the most similar memory access patterns, VISTA further detects data locality among warps and prioritizes those more likely to reuse data for execution. This is achieved by leveraging both the history of application behavior and the runtime memory access patterns of the warps. To achieve this, incorporates a warp locality tracker built on a lightweight learning-based model, the Integer Support Vector Machine (ISVM). The warp locality tracker leverages the ISVM model to analyze the history of program counters from memory instructions and tracks the runtime behavior of warp accesses. This unit also generates a hashed vector that represents the memory pattern of each SM.

In this paper, we make the following contributions:

- We observe that previous scheduling methods fall short as they fail to leverage data sharing among non-adjacent CTAs or warps.
- We introduce VISTA, a novel approach that harnesses data locality among adjacent and non-adjacent CTAs and warps, incorporating dynamic locality-aware CTA and warp schedulers.
- Results show that VISTA improves performance by 48.1% and reduces energy consumption by 51.8% for memory-intensive GPGPU applications, compared to the baseline, with negligible hardware overhead.

II. BACKGROUND AND MOTIVATION

Figure 1 provides an overview of the GPU architecture. GPUs consist of multiple SMs, with a global Giga Thread scheduler that distributes CTAs to SMs using a round-robin policy. Within each SM, threads execute in the form of warps (a group of threads executing in lock-step), which are scheduled by warp schedulers. Each SM comprises various key units, such as Streaming Processors (SP) grouped in four subcores, Load Store Units (LD/ST), warp schedulers, and on-chip storages (e.g., register files and L1D cache). SMs are connected to the memory hierarchy and off-chip DRAM through a network. The memory hierarchy includes L2 cache banks, memory controllers, and DRAM channels.

TABLE I
PERCENTAGE OF DATA SHARING IN GPGPU BENCHMARKS.

Benchmark suite	Intra-SM Data sharing%	Inter-SM Data sharing%
Ispass-2009 [29]	81%	68%
Rodinia [30]	65%	53%
LonestarGPU [31]	85%	72%
PolyBench [32]	89%	78%
Parboil [33]	63%	52%
Pannotia [34]	84%	71%
GraphBench [35]	87%	70%
SDK [36]	78%	69%
DeepBench [37]	92%	81%

The memory hierarchy, combined with high TLP, is designed to meet the extensive memory demands of memory-intensive GPGPU applications.

A major bottleneck in memory-intensive GPGPU applications is the large number of memory accesses generated by thousands of threads, leading to frequent cache contention and evictions. An effective approach to mitigate this issue is to preserve data locality by retaining shared data for reuse across threads, which reduces the frequency of memory requests [24]. Thus, closely related CTAs are assigned to the same SM, significantly improving memory system performance.

We analyze the extent of data sharing across various levels and investigate sharing patterns in memory-intensive GPGPU applications. For more details about our methodology, refer to §IV. To illustrate the prevalence of such accesses in memory-intensive GPGPU applications, we measure the fraction of memory accesses shared within an SM and across SMs in GPGPU applications. Table I summarizes these results. We define a memory access as shared whenever a thread accesses an address previously accessed by another thread. Our observations show that all applications exhibit significant inter-SM and intra-SM data sharing, with 68.2% and 80.4% of memory accesses shared, respectively.

To analyze data-sharing patterns, we quantify the shared data among threads based on SM distance, as shown in Figure 2. Our results show that 95% to 78% of the shared data occurs as the SM distance increases from 0 to 40, highlighting significant data sharing between non-adjacent CTAs. We further observe that around 80% of the total data sharing within an SM occurs between non-adjacent warps.

While previous studies have confirmed notable data sharing in GPUs [1], [3], [25]–[28], they have mainly focused on adjacent CTAs or warps, or on sharing across SMs. These approaches fail to fully exploit the potential of dynamically scheduling both warps and CTAs. In contrast, our findings emphasize the importance of data sharing among non-adjacent CTAs and warps, which can be achieved by proactively scheduling these high-locality CTAs and warps closer together.

To summarize, the increased frequency of memory requests leads to higher delays and power consumption within the GPU memory subsystem. Even in high TLP scenarios, where numerous parallel threads generate thousands of memory accesses, significant contention occurs which further exacerbates delays, especially due to queuing delays in memory resources [25], [27]. In this work, we build upon the key observation that non-adjacent CTAs and warps share a sub-

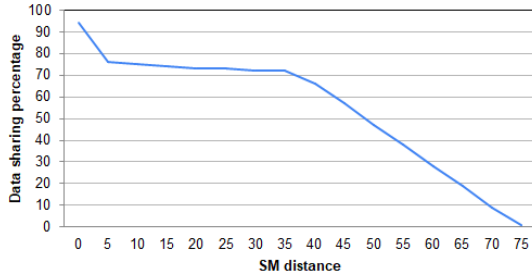


Fig. 2. Data-sharing pattern with respect to SM distance.

stantial amount of data. Existing scheduling mechanisms fail to simultaneously leverage data locality within SMs and data sharing across CTAs, missing opportunities to fully optimize data sharing in GPUs. Our goal is to propose a proactive scheduling mechanism for GPUs that addresses these gaps by 1) detecting and utilizing data sharing among non-adjacent warps within each SM while preserving cache locality, and 2) discovering and leveraging data sharing among non-adjacent CTAs. We believe that integrating these two features in scheduling improves overall performance and reduces energy consumption in GPUs.

III. VISTA

We propose VISTA, a smart locality-aware scheduler for GPUs, designed to dynamically detect and exploit fine-grained data locality patterns at both CTA and warp levels through intelligent scheduling decisions. By doing so, VISTA optimizes memory management, reduces memory accesses, and significantly enhances performance and energy efficiency.

First, VISTA dynamically identifies data locality among CTAs by running two warps of unscheduled CTAs on idle SMs. By executing the first few memory instructions (typically 4) of two representative warps, VISTA tracks the memory behavior of their associated CTAs. It then classifies CTAs based on locality and assigns them to the SM with the most similar memory access pattern. Once CTAs are grouped and assigned to the SMs with the most similar memory access patterns, VISTA further enhances memory usage by dynamically detecting data locality among warps using a lightweight learning-based model. The warp scheduler is modified to prioritize warps that are more likely to reuse data.

VISTA consists of two key units: the warp locality tracker and the CTA locality tracker. First, we describe the warp locality tracker and the process of extracting identifiers for memory patterns of SMs (§III-A). Second, we describe how the CTA locality tracker (§III-B) classifies incoming CTAs based on the similarity of their memory access pattern to previously observed patterns in each SM.

A. Warp Locality Tracker

Each SM contains a warp locality tracker that utilizes a lightweight learning-based model, an Integer Support Vector Machine (ISVM) [38], to predict the memory behavior of the assigned warps. By analyzing the memory pattern of warps assigned to the SM, the warp locality tracker (1) identifies which warps have a higher likelihood of maintaining data

locality within the SM and gives these warps higher execution priority, and (2) creates a hashed vector representing the SM's memory pattern (Pattern SM), used to detect incoming CTAs with similar memory access patterns.

The warp locality tracker is composed of two main components: a prediction unit, which forecasts memory behavior and data locality, and a priority unit, which assigns execution priority to warps based on these predictions.

1) *Prediction Unit*: Figure 3 and Figure 4 depict the prediction unit structure and its flowchart, respectively. To predict the memory behavior among warps and CTAs, we (1) utilize the ISVM model that considers the history of program counters from memory instructions (PC_{mem}), and (2) consider the runtime behavior through tracking the history of warps accesses. Consistent with prior research [38], [39], we found it unnecessary to consider a full instruction history. Consequently, our designed ISVM model tracks the 16 recent warp accesses working on selected features extracted from recent PC_{mem} and data sharing percentage history. We train the ISVM model by profiling data (logged PC_{mem} and data sharing percentage) from a diverse set of applications, focusing on 20% of the representative GPGPU kernels.

Warp locality prediction. Warp locality prediction consists of two key units: the PC history and the sharing history units. The PC history unit includes a hashed PC table, a weight table, and a hashing mechanism (F_{hash} in Figure 3). The hashed PC table stores 16 hashed PCs, each pointing to an entry in the sharing history unit. The hashing mechanism creates a 4-bit vector by XOR-ing 16-bit chunks of the 64-bit PC_{mem} . The weight table holds trained weights for each hashed PC pattern, which quantifies the contribution of each PC to the warp locality score. The sharing history unit monitors which warp accesses each hashed memory address. It is organized as a 16-entry table, with each entry representing a hashed memory address. Each entry contains 64 elements, one for each ready warp, and tracks up to 16 memory accesses per warp. To calculate the sharing score for each warp, the total number of memory accesses issued by that warp is divided by the total number of accesses, producing a score between zero and one ($F_{sharing_score}$ in Figure 3).

When a new PC_{mem} is encountered, it is hashed and used to index the hashed PC table. If the pattern is new, a corresponding entry is allocated in the sharing history unit. For previously seen patterns, the sharing history unit is updated. The weights and sharing scores (S_i in Figure 3) for each hashed PC pattern are multiplied to generate the overall warp locality score (FS_i).

SM memory pattern. The pattern detector unit selects the four hashed memory PC with the highest sharing amount. The sharing amount is determined by counting how many accesses are issued by different warps. To represent the overall memory behavior of each SM (SM memory pattern), the pattern detector generates a compact 16-bit signature vector consisting of four 4-bit hashed memory PCs, indicating which hashed memory addresses are most frequently accessed.

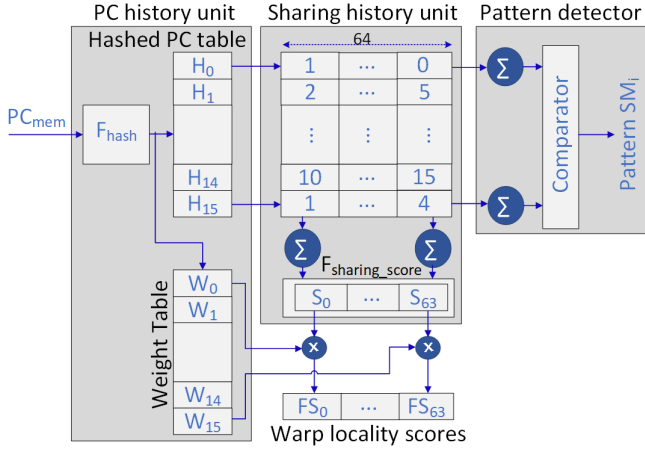


Fig. 3. Structure of prediction unit in warp locality tracker.

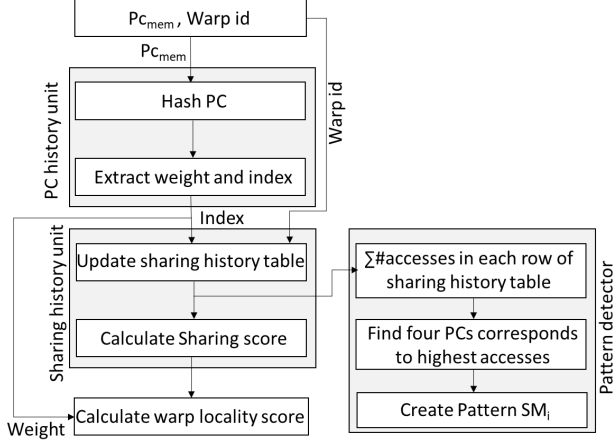


Fig. 4. Flowchart of prediction unit in warp locality tracker.

2) **Priority Unit:** Based on the detected memory access patterns, the priority unit selects the warps with the highest potential for maintaining data locality, as shown in Figure 5. It does this by evaluating eligible warps as execution candidates and sorting them according to their warp locality scores (FS_i in Figure 3). Warps with the highest locality scores are given priority for execution. In cases where multiple warps have the same score, the priority unit breaks the tie by selecting the warp with the lowest ID.

B. CTA Locality Tracker

We enhance the GPU's CTA scheduler with a CTA locality tracker, which aims to assign each incoming CTA to the SM that exhibits the most similar memory access pattern. The CTA locality tracker employs Locality-Sensitive Hashing (LSH) for similarity detection. LSH is an efficient online clustering technique known for its low overhead and high clustering

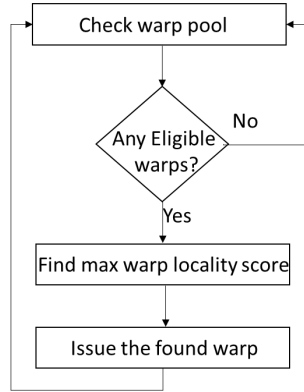


Fig. 5. Priority flowchart.

accuracy. Figure 6 shows the LSH-based similarity checker. It extracts the memory access pattern of a CTA, compares it with the memory patterns of SMs, and selects the SM with the closest matching memory pattern for the CTA assignment.

CTA access pattern.

To capture the memory access pattern of a CTA, the CTA locality tracker executes its first warp and one randomly selected warp until four memory accesses are issued in these two warps. Prior research suggests that observing four memory accesses provides a good approximation of the overall memory behavior of a warp [39]. These warps are executed on idle subcores, as there are typically idle subcores in GPUs [40].

Using LSH, the accessed memory addresses are projected onto a lower-dimensional binary space via multiple hash functions, forming a 16-bit binary locality signature, similar to the way ISVM handles memory addresses in the warp locality tracker (F_{hash} in Figure 3). Each warp's memory accesses are hashed into this 16-bit vector, and by XOR-ing the vectors from the two executed warps, the CTA locality tracker derives a 16-bit vector that represents the CTA's access pattern.

Assigning CTA.

Figure 7 shows how VISTA assigns CTAs to the best match SM. Once the CTA access pattern is extracted, the CTA locality tracker compares it with the memory patterns of all SMs to find the best match. The similarity between the CTA's memory pattern and each SM's pattern is measured by calculating the distance between their respective signature vectors. The CTA is then assigned to the SM with the smallest distance. In cases where multiple SMs exhibit similar distances, the tracker resolves this by selecting the SM with the lowest load.

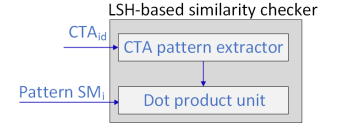


Fig. 6. CTA locality tracker structure.

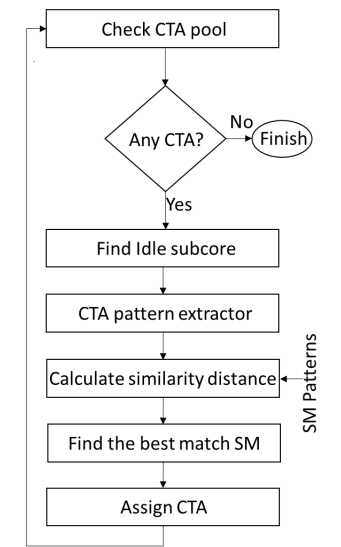


Fig. 7. CTA locality tracker flowchart.

IV. METHODOLOGY

Simulation. We evaluate VISTA using Accel-Sim V1.0 [41] and Accel-Watch [42] to model an NVIDIA Volta QV100 GPU [43]. Table II shows simulation parameters. We modify the CTA scheduler and warp scheduler to provide the architectural supports in our proposal (§III). The reported results fully incorporate the overheads introduced by VISTA.

Workloads. We evaluate our proposal under a wide range of standard GPGPU applications from Rodinia [30], LonestarGPU [31], Ispass2009 [29], Pannotia [34], and GraphBench [35] benchmark suites (Table III). We selected representative benchmarks from various domains, 40 applications for training, and 20 distinct applications for testing, achieving an average data locality prediction accuracy of 85% on the training set and 80% on the test set. We further validate the robustness of the model by evaluating it on memory-intensive workloads excluded from the training set.

TABLE II
SIMULATION CONFIGURATION.

Parameter	Value
Number of SM	80
Core clock	1530 MHz
Scheduler	Greedy-then-Oldest
Number of scheduler per SM	4
Number of threads per SM	2048
Register file size per SM	65536
Unified cache	adaptive, 128KB, 256-way, 128B-line, 4-bank, 28-cycle
L1 instruction cache	128KB, 16-way, 128B-line
Constant cache	64KB, 8-way, 64B-line
L2 cache	96 KB/subpartition, 24-way, 128B-line, 32-set, 160-cycle

TABLE III
BASELINE BENCHMARK SUITE.

Application	Abbreviation
Breadth-first search	BFS [35]
B+tree search	BTR [30]
Hotspot	HSP [30]
Needleman-wunsch	NW [30]
LU decomposition	LUD [30]
Speckle Reducing Anisotropic Diffusion	SRAD [30]
Ray tracing	RAY [29]
StoreGPU	STO [29]
Components Labeling	CCL [31]
Particle filter	PF [30]
Page rank	pr [34]

Comparison points and metrics. We compare VISTA with the following systems: (1) Baseline: an NVIDIA Volta QV100 [43], configured as in Table II; (2) CTA-VISTA: a variant of VISTA without the warp locality tracker unit; (3) Warp-VISTA: a version of VISTA without the CTA locality tracker unit; (4) CCDS [3]: a state-of-the-art data-sharing-aware SM coordination system; and (5) CCWS [15]: a locality-aware warp scheduler. We report the instruction per cycle (IPC) and the energy consumption as the comparison metrics.

V. EVALUATION

Performance and energy analysis. Figure 8 presents the performance of all comparison points, normalized to the Baseline. We draw two key observations. First, all evaluated designs (VISTA, CTA-VISTA, Warp-VISTA, CCDS [3], and CCWS [15]) outperform the Baseline for memory-intensive GPGPU applications, delivering average IPC gains of 48.1%, 33.8%, 24.1%, 21.2%, and 12.4%, respectively. Second, applications with high memory intensity and non-adjacent data sharing patterns, such as PF [30], benefit significantly more from VISTA compared to the other designs.

Figure 9 reports the energy consumption normalized to Baseline. We observe that VISTA, CTA-VISTA, Warp-VISTA,

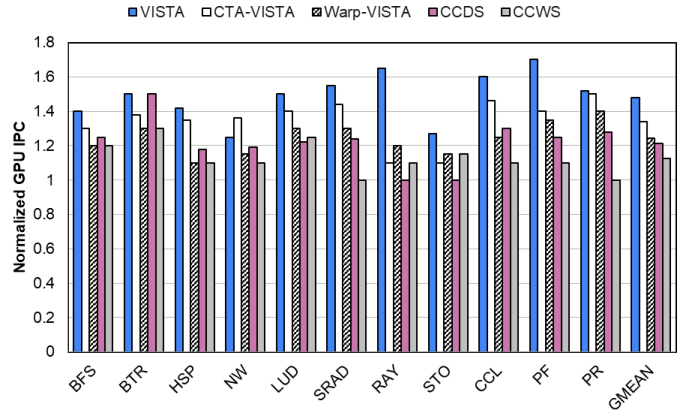


Fig. 8. Total GPU IPC of VISTA, CTA-VISTA, Warp-VISTA, CCDS, and CCWS normalized to Baseline.

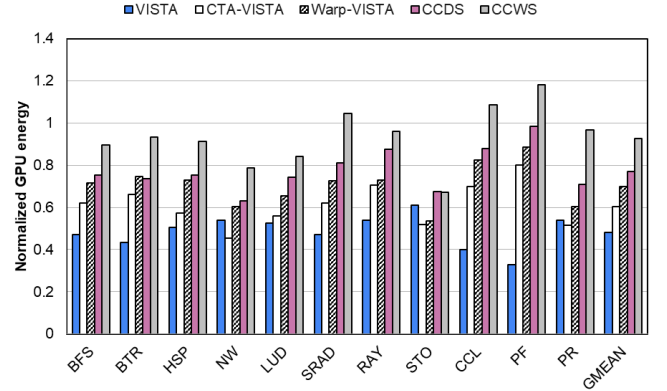


Fig. 9. Total GPU energy consumption of VISTA, CTA-VISTA, Warp-VISTA, CCDS, and CCWS normalized to Baseline.

CCDS [3], and CCWS [15] reduce the total energy consumption of memory-intensive GPGPU applications by an average of 51.8%, 39.7%, 30.1%, 22.8%, and 7.5%, respectively.

Source of benefits. To better understand the advantages of our proposed approach, we analyze how VISTA impacts the L1D/L2 cache hit rate. Figure 10 reports the L1D/L2 hit rate improvements of VISTA normalized to Baseline. We observe that VISTA improves the L1D and L2 hit rates by 53.9% and 48.7%, respectively, reduces memory stalls by 50.4%, and decreases execution stalls by 46.3%, compared to Baseline for memory-intensive GPGPU applications. These results demonstrate that VISTA effectively improves data locality and execution efficiency, thereby reducing both memory and scheduling stalls.

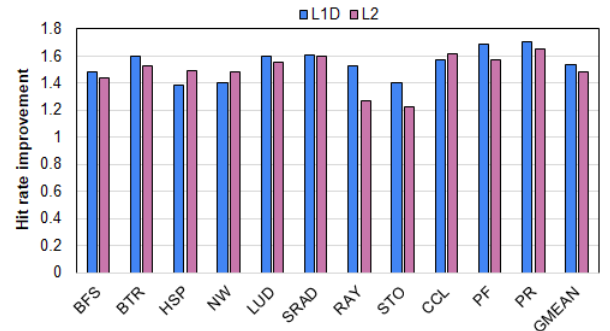


Fig. 10. L1D/L2 Hit rate improvement of VISTA normalized to Baseline.

Hardware Overhead. VISTA introduces two key components: a warp locality tracker for each SM and a CTA locality tracker for the entire GPU. These components are evaluated for area and power overheads using a hybrid modeling approach that combines detailed architectural simulations with analytical modeling for the unchanged portions of the GPU. The warp locality tracker consists of four main units. (1) Hashed PC table and sharing history table consist of 16 and 64×16 entries (4 bits per entry), requiring 8B and 128B of storage, respectively. (2) Weight table, F_sharing-score, and warp locality score with 16, 64, and 64 entries, respectively, where each element is 8 bits; The total storage for these components is 16 B, 64 B, and 64 B, respectively. (3) The logic unit includes a hash function and a total of 2,112 4-bit adders, comprising 64×16 adders for the hashed PC and another 64×16 for sharing history operations, and 64 additional 8-bit adders for score computation. (4) The CTA locality tracker includes 16 multipliers and 15 adders, working on 16-bit operands.

Area. The storage components for the warp and CTA locality trackers are modeled using CACTI V7.0 [44] for 22nm technology and scaled to a 12nm process node, consistent with the fabrication technology of the NVIDIA Volta V100 GPU. The area overhead including wiring is less than 3% of the total GPU area (815 mm² for Volta V100).

Power. The power consumption of these components is estimated using a synthesized Hardware Description Language (HDL) model evaluated with Synopsys Design Compiler and a 45nm NanGate open cell library. Accel-Wattch is used to analyze both dynamic and static power. The estimated 9.1 W power corresponds to less than 2% of the total GPU power.

Time. The warp locality tracker computes the memory pattern within 7 cycles and the warp locality score in 10 cycles. Some operations overlap (PC history unit and sharing history unit), while others execute in parallel (pattern detector and final computation for warp locality score). The CTA locality tracker extracts the CTA memory access pattern in 7 cycles and performs the dot product in 5 cycles, resulting in 12 cycles.

VI. RELATED WORK

We propose a smart locality-aware GPU scheduler, VISTA, that dynamically identifies data locality patterns at both the CTA and warp levels to make intelligent scheduling decisions. VISTA proactively detects inter-CTA locality by assigning CTAs to the SM with the most similar memory behavior. Within each SM, VISTA further identifies inter-warp locality and prioritizes warps that preserve data locality. We have extensively compared VISTA to different locality-aware mechanisms in §V. In this section, we discuss other related work that aims to tackle GPU memory bottleneck: Locality-aware scheduling, data sharing, and smart resource management.

Locality-aware scheduling and data sharing mechanism. Several warp schedulers aim to improve L1D cache performance by prioritizing warps likely to produce cache hits and by limiting the number of active warps to reduce cache thrashing [?], [11]–[15], [17]–[21]. Locality-aware CTA schedulers [14], [22], [23], [45] improve data locality by

clustering nearby CTAs on the same SM [9], [10] or restricts the number of active CTAs [14], [23] to balance TLP and reduce cache contention. However, many of these approaches rely on compile-time analysis, which may be inaccurate for workloads with irregular or input-dependent memory behavior.

Other techniques improve data reuse by modifying cache behavior through bypassing [46], buffering [47], pinning [48], and inter-core cooperation [1], [3], [25]–[27]. None of them jointly consider locality across both CTAs and warps in a unified, proactive manner. Some research [28] locates the on-chip shared memory in the global memory and uses a unified cache to service both L1D and shared memory accesses. VISTA is orthogonal to such techniques; instead, it proactively schedules CTAs and warps by considering data sharing among non-adjacent CTAs and warps.

Smart resource management. Machine learning (ML) approaches [49], [50] are widely used in different areas, such as CPU resource management. Some use RNNs to model memory access patterns [51], a perceptron-based method for prefetch filtering [52], LSTMs and attention mechanisms to address cache replacement as a sequence prediction problem [38], and reinforcement learning for designing cost-effective cache replacement policies [53]. In the context of CPU-GPU systems, some propose [54] an ML-based prefetching approach for unified virtual memory to mitigate on-demand paging overhead. While ML has gained traction in CPU resource management, its application to GPU resource management remains underexplored.

VII. CONCLUSION

VISTA, a smart locality-aware GPU scheduler, addresses key memory bottlenecks in memory-intensive GPGPU applications by exploiting data sharing across both adjacent and non-adjacent CTAs and warps. Unlike prior approaches, VISTA dynamically identifies data locality patterns at both the CTA and warp levels and proactively schedules computations to preserve locality. This strategy significantly reduces redundant memory accesses, leading to a 48.1% improvement in performance and a 51.8% reduction in energy consumption compared to the Baseline. These results highlight the potential of VISTA as a lightweight and effective solution for optimizing GPU performance and energy efficiency, making it a promising solution for future memory-intensive applications and high-throughput computing environments.

VIII. ACKNOWLEDGMENT

The authors would like to express their gratitude to the anonymous reviewers for their valuable feedback. This work is part of the project PID2023-146511NB-I00 funded by the Spanish Ministry of Science, Innovation and Universities MICIU /AEI /10.13039/501100011033 and EU ERDF. This publication is promoted by the Barcelona Zettascale Laboratory, backed by the Ministry for Digital Transformation and of Public Services, within the framework of the Recovery, Transformation, and Resilience Plan – funded by the European Union – NextGenerationEU.

REFERENCES

- [1] S. Dubliss, V. Nagarajan, and N. Topham, "Cooperative caching for GPUs," *ACM TOCS*, 2016.
- [2] M. Sadrosadati, A. Mirhosseini, A. Hajiabadi, S. B. Ehsani, H. Falahati, H. Sarbazi-Azad, M. Drumond, B. Falsafi, R. Ausavarungnirun, and O. Mutlu, "Highly concurrent latency-tolerant register files for GPUs," *ACM TOCS*, 2021.
- [3] H. Falahati, M. Sadrosadati, Q. Xu, J. Gómez-Luna, B. S. Latibari, H. Jeon, S. Hesaabi, H. Sarbazi-Azad, O. Mutlu, M. Annavaram *et al.*, "Cross-core data sharing for energy-efficient gpus," *ACM TACO*, 2022.
- [4] M. Sadrosadati, S. B. Ehsani, H. Falahati, R. Ausavarungnirun, A. Tavakkol, M. Abaee, L. Orosa, Y. Wang, H. Sarbazi-Azad, and O. Mutlu, "ITAP: Idle-time-aware power management for GPU execution units," *ACM TACO*, 2019.
- [5] G. Koo, H. Jeon, and M. Annavaram, "Revealing Critical Loads and Hidden Data Locality in GPGPU Applications," in *IISWC*, 2015.
- [6] H. Falahati, M. Abdi, A. Baniasadi, and S. Hessabi, "Power-efficient prefetching on GPGPUs," *The Journal of Supercomputing*, 2015.
- [7] H. Falahati, M. Abdi, A. Baniasadi, and S. Hessabi, "ISP: Using Idle SMs in Hardware-based Prefetching," in *CADS*, 2013.
- [8] T. G. Rogers, M. O'Connor, and T. M. Aamodt, "Cache-conscious wavefront scheduling," in *ISCA*, 2012.
- [9] L.-J. Chen, H.-Y. Cheng, P.-H. Wang, and C.-L. Yang, "Improving GPGPU Performance via Cache Locality Aware Thread Block Scheduling," *IEEE CAL*, 2017.
- [10] A. Tabbakh, M. Annavaram, and X. Qian, "Power Efficient Sharing-Aware GPU Data Management," in *IPDPS*, 2017.
- [11] V. Narasiman, M. Shebanow, C. J. Lee, R. Miftakhutdinov, O. Mutlu, and Y. N. Patt, "Improving GPU performance via large warps and two-level warp scheduling," in *MICRO*, 2011.
- [12] R. Ausavarungnirun, S. Ghose, O. Kayiran, G. H. Loh, C. R. Das, M. T. Kandemir, and O. Mutlu, "Exploiting Inter-Warp Heterogeneity to Improve GPGPU Performance," in *PACT*, 2015.
- [13] A. Jog, O. Kayiran, A. Pattnaik, M. T. Kandemir, O. Mutlu, R. Iyer, and C. R. Das, "Exploiting core criticality for enhanced GPU performance," in *SIGMETRICS*, 2016.
- [14] O. Kayiran, A. Jog, M. T. Kandemir, and C. R. Das, "Neither More nor Less: Optimizing Thread-level Parallelism for GPGPUs," in *PACT*, 2013.
- [15] T. G. Rogers, M. O'Connor, and T. M. Aamodt, "Cache-conscious wavefront scheduling," in *MICRO*, 2012.
- [16] A. Sethia, D. A. Jamshidi, and S. Mahlke, "Mascar: Speeding up GPU Warps by Reducing Memory Pitstops," in *HPCA*, 2015.
- [17] A. Jog, O. Kayiran, N. Chidambaram, A. K. Mishra, M. T. Kandemir, O. Mutlu, R. Iyer, and C. R. Das, "OWL: Cooperative Thread Array Aware Scheduling Techniques for Improving GPGPU Performance," in *ASPLOS*, 2013.
- [18] D. Li, M. Rhu, D. R. Johnson, M. O'Connor, M. Erez, D. Burger, D. S. Fussell, and S. W. Keckler, "Priority-Based Cache Allocation in Throughput Processors," in *HPCA*, 2015.
- [19] M. Gebhart, D. R. Johnson, D. Tarjan, S. W. Keckler, W. J. Dally, E. Lindholm, and K. Skadron, "Energy-efficient Mechanisms for Managing Thread Context in Throughput Processors," in *ISCA*, 2011.
- [20] A. Sethia and S. Mahlke, "Equalizer: Dynamic Tuning of GPU Resources for Efficient Execution," in *MICRO*, 2014.
- [21] S. Dubliss, V. Nagarajan, and N. Topham, "Poise: Balancing thread-level parallelism and memory system performance in GPUs using machine learning," in *HPCA*, 2019.
- [22] L. Wang, X. Zhao, D. Kaeli, Z. Wang, and L. Eeckhout, "Intra-cluster coalescing to reduce gpu noc pressure," in *IPDPS*, 2018.
- [23] J. Wang, N. Rubin, A. Sidelnik, and S. Yalamanchili, "Laperm: Locality aware scheduler for dynamic parallelism on gpus," in *ISCA*, 2016.
- [24] A. Mohammadpur-Fard, S. Darabi, H. Falahati, N. Mahani, and H. Sarbazi-Azad, "Exploiting direct memory operands in gpu instructions," *IEEE CAL*, 2024.
- [25] N. Nematollahi, M. Sadrosadati, H. Falahati, M. Barkhordar, and H. Sarbazi-Azad, "Neda: Supporting direct inter-core neighbor data exchange in GPUs," *IEEE CAL*, 2018.
- [26] M. M. Keshtegar, H. Falahati, and S. Hessabi, "Cluster-based approach for improving graphics processing unit performance by inter streaming multiprocessors locality," *IET Computers & Digital Techniques*, 2015.
- [27] N. Nematollahi, M. Sadrosadati, H. Falahati, M. Barkhordar, M. P. Drumond, H. Sarbazi-Azad, and B. Falsafi, "Efficient nearest-neighbor data sharing in GPUs," *ACM TACO*, 2020.
- [28] S. Darabi, E. Yousefzadeh-Asl-Miandoab, N. Akbarzadeh, H. Falahati, P. Lotfi-Kamran, M. Sadrosadati, and H. Sarbazi-Azad, "OSM: Off-chip shared memory for GPUs," *IEEE TPDS*, 2022.
- [29] A. Bakhoda, G. L. Yuan, W. W. Fung, H. Wong, and T. M. Aamodt, "Analyzing cuda workloads using a detailed gpu simulator," in *ISPASS*, 2009.
- [30] S. Che, M. Boyer, J. Meng, D. Tarjan, J. W. Sheaffer, S.-H. Lee, and K. Skadron, "Rodinia: A benchmark suite for heterogeneous computing," in *IISWC*, 2009.
- [31] M. Burtcher, R. Nasre, and K. Pingali, "A quantitative study of irregular programs on GPUs," in *IISWC*, 2012.
- [32] S. Grauer-Gray, L. Xu, R. Searles, S. Ayalasomayajula, and J. Cavazos, "Auto-tuning a high-level language targeted to GPU codes," in *IEEE InPar*, 2012.
- [33] J. A. Stratton, C. Rodrigues, I.-J. Sung, N. Obeid, L.-W. Chang, N. Anssari, G. D. Liu, and W.-m. W. Hwu, "Parboil: A revised benchmark suite for scientific and commercial throughput computing," *Center for Reliable and High-Performance Computing*, 2012.
- [34] S. Che, B. M. Beckmann, S. K. Reinhardt, and K. Skadron, "Pannotia: Understanding irregular gpgpu graph applications," in *IISWC*, 2013.
- [35] Xu, Qiumin and Jeon, Hyeran and Annavaram, Murali, "Graph processing on GPUs: Where are the bottlenecks?" in *IISWC*, 2014.
- [36] NVIDIA. (2009) Cuda sdk 2.3. [Online]. Available: <https://developer.nvidia.com/cuda-toolkit-23-downloads>
- [37] Baidu. (2017) Deepbench. [Online]. Available: <https://github.com/baidu-research/DeepBench>
- [38] Z. Shi, X. Huang, A. Jain, and C. Lin, "Applying deep learning to the cache replacement problem," in *MICRO*, 2019.
- [39] S. Mostofi, H. Falahati, N. Mahani, P. Lotfi-Kamran, and H. Sarbazi-Azad, "Snake: A variable-length chain-based prefetching for gpus," in *MICRO*, 2023.
- [40] A. Barnes, F. Shen, and T. G. Rogers, "Mitigating gpu core partitioning performance effects," in *HPCA*, 2023.
- [41] M. Khairy, Z. Shen, T. M. Aamodt, and T. G. Rogers, "Accel-sim: An extensible simulation framework for validated gpu modeling," in *ISCA*, 2020.
- [42] V. Kandiah, S. Peverelle, M. Khairy, J. Pan, A. Manjunath, T. G. Rogers, T. M. Aamodt, and N. Hardavellas, "Accelwattch: A power modeling framework for modern gpus," in *MICRO*, 2021.
- [43] NVIDIA. (2017) Volta architecture whitepaper. [Online]. Available: <https://images.nvidia.com/content/volta-architecture/pdf/volta-architecture-whitepaper.pdf>
- [44] S. J. Wilton and N. P. Jouppi, "CACTI: An enhanced cache access and cycle time model," *IEEE JSSC*, 1996.
- [45] S. Darabi, N. Mahani, H. Baxishi, E. Yousefzadeh-Asl-Miandoab, M. Sadrosadati, and H. Sarbazi-Azad, "NURA: A framework for supporting non-uniform resource accesses in GPUs," *ACM PoMACS*, 2022.
- [46] X. Xie, Y. Liang, Y. Wang, G. Sun, and T. Wang, "Coordinated static and dynamic cache bypassing for gpus," in *HPCA*, 2015.
- [47] H. Choi, J. Ahn, and W. Sung, "Reducing off-chip memory traffic by selective cache management scheme in gpgpus," in *ACM GPGPUs*, 2012.
- [48] G. Koo, Y. Oh, W. W. Ro, and M. Annavaram, "Access pattern-aware cache management for improving data utilization in gpu," in *ISCA*, 2017.
- [49] H. Falahati, M. Peyro, H. Amini, M. Taghian, M. Sadrosadati, P. Lotfi-Kamran, and H. Sarbazi-Azad, "Data-Aware compression of neural networks," *IEEE CAL*, 2021.
- [50] Sze, Vivienne and Chen, Yu-Hsin and Yang, Tien-Ju and Emer, Joel S, "Efficient processing of deep neural networks," *Synthesis Lectures on Computer Architecture*, 2020.
- [51] M. Hashemi, K. Swersky, J. Smith, G. Ayers, H. Litz, J. Chang, C. Kozyrakis, and P. Ranganathan, "Learning memory access patterns," in *PMLR*, 2018.
- [52] E. Bhatia, G. Chacon, S. Pugsley, E. Teran, P. V. Gratz, and D. A. Jiménez, "Perceptron-based prefetch filtering," in *ISCA*, 2019.
- [53] S. Sethumurugan, J. Yin, and J. Sartori, "Designing a cost-effective cache replacement policy using machine learning," in *HPCA*, 2021.
- [54] X. Long, X. Gong, B. Zhang, and H. Zhou, "Deep learning based data prefetching in cpu-gpu unified virtual memory," *Elsevier JPDC*, 2023.